Notes 2 - R Guide

Math 3190 - Fundamentals of Data Science

Rick Brown

Southern Utah University

Section 1

Introduction to **R** and RStudio

Introduction

R is a powerful and popular programming language for statistical computing and data analysis. It provides a wide range of tools and packages for data manipulation, visualization, and modeling. **R** has a rich ecosystem with a vast community contributing to its growth, making it an excellent choice for data scientists, statisticians, and researchers.

Introduction

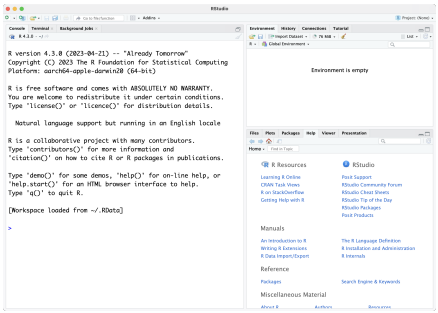
While **R** was developed by statisticians for statisticians for statistical analysis, it has evolved to be a full-fledged programming language comparable to C, Python, Java, etc., though it is arguably still best used for data analysis tasks since many tasks are much more efficient in other languages. In this class, we will explore the syntax of **R** and the most commonly used functionality of the language.

Installing R and RStudio

In the Introduction module on Canvas, there are some documents about installing **R** and **RStudio**. Be sure to follow those instructions to get the programs installed on your system. In this class, we will use **RStudio** exclusively once it is installed.

R First Impressions

When you first open **RStudio**, you will be presented with several windows, the main one being the **R** console, as shown below.



If yours does not look like this, then **something is wrong**!

R Console

You can interact with the console directly by typing in commands. For instance, if you type

```
1+1
```

into the console and it will give

```
[1] 2
```

as an output. The [1] on the left indicates that the output has only 1 value. I encourage you to play around with the console for a few minutes.

R Operations

You can perform all of the usual arithmetic operations in **R**. This includes

- + for addition.
- − for subtraction.
- ∗ for multiplication.
- / for division.
- ^ for exponents.

R will do the order of operations: PEMDAS, so be sure to use parentheses properly.

```
2-6/2+5
```

```
[1] 4
```

```
(2-6)/2+5
```

```
[1] 3
```


R Operations

You can also use <, >, <=, >=, or == for **R** to check if one expression is less than, greater than, at most, at least, or exactly equal to another expression.

R will return either TRUE or FALSE, which is known as a Boolean data type.

```
3 > 1
```

```
[1] TRUE
```

```
-5 < -4
```

```
[1] TRUE
```

```
5 <= 5
```

```
[1] TRUE
```

```
5 == 5
```

```
[1] TRUE
```


R Data Types

Everything that you enter in **R** will have a data type. The primary data types in **R** are:

- **numeric**: data that are numbers. This will be the default for any number entered.
- **integer**: data that are numbers without decimal points. You can force a number to be stored as an integer by typing a capital "L" after the number. Example: 14L.
- **character**: data that are strings, which can include numbers and letters. This will be the default for anything entered in quotes (either single quotes or double quotes) like 'yellow' or "yellow".

R Data Types

- **factor**: data that are categorical. **R** thinks of these just in terms of categories.
- **logical**: true or false. Also known as Boolean. **R** will convert to logical if you are making a comparison like we did a couple slides ago.

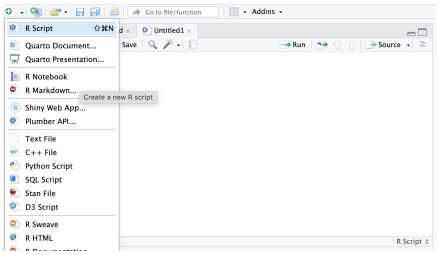
The type of variable is important when working in **R**. If you have the wrong data type, then you may get errors. The code below will give an error.

```
"1" + "1"
```

```
Error in "1" + "1" : non-numeric argument to binary
operator
```

R Scripts

When programming in **R**, it is good practice to type all of your code in an **R** script. You can access a new script by clicking on the green plus icon at the top of the **RStudio** menu and then clicking on "R Script".



This way, you can save all your code to easily rerun it later.

R Commenting

You can comment out code or other words in **R** scripts with the # key. Anything on the same line after # will not be run.

```
x <- 5
# print(x)
x + 2
```

```
[1] 7
```

In the output above, the value of `x` was not printed since the `print(x)` command was commented out.

Built-In **R** Functions

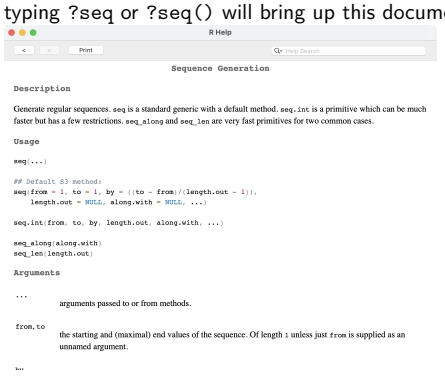
There are many functions that **R** comes pre-loaded with. To use a function in **R**, you must type the name of the function, then put the arguments of the function in parentheses. Some examples are

- `c()` to create a data vector.
- `mean()` to compute the average of a vector.
- `sd()` to compute the standard deviation of a vector.
- `length()` to compute the number of elements in a vector.
- `rep()` to repeat a value, string, or vector.
- `seq()` to create a sequence of values.
- `summary()` to compute some summary statistics for a vector.
- `cos()` to compute the cosine of a value.
- `exp()` to compute the number e raised to a power.
- `log()` to compute the logarithm (natural log is default) of a number.

There are hundreds of built-in **R** functions.

More on Functions

If you are ever in doubt about what a function does or what inputs it takes, you can always type a question mark in front of the function. For instance, typing `?seq` or `?seq()` will bring up this documentation:


R Help

Sequence Generation

Description

Generate regular sequences. seq is a standard generic with a default method. seq.int is a primitive which can be much faster but has a few restrictions. seq_along and seq_len are very fast primitives for two common cases.

Usage

```
seq(...)

## Default S3 method:
seq(from = 1, to = 1, by = ((to - from)/(length.out - 1)),
    length.out = NULL, along.with = NULL, ...)

seq.int(from, to, by, length.out, along.with, ...)

seq_along(along.with)
seq_len(length.out)
```

Arguments

...	arguments passed to or from methods.
from, to	the starting and (maximal) end values of the sequence. Of length 1 unless just from is supplied as an unnamed argument.

Creating a Numeric Variable

It is remarkably useful to create variables for use in **R**. For example, if I have a small data set containing the values [1, 5, 3, 6], then I can input that into **R** and give that data set a name.

```
x <- c(1, 5, 3, 6)
```

The code above will create a new variable called `x` and then if I type `x`, it will dispaly what is contained in `x`.

```
x
```

```
[1] 1 5 3 6
```

Then I can tell **R** to report how many elements are in `x` using the `length()` function, for instance.

```
length(x)
```

```
[1] 4
```


Creating a Character Variable

Variables do not have to be numbers. They can also be words (called "strings"). Just be sure to put strings in quotes.

```
y <- c("This", "Is", "A", "String", "Variable")
y
```

```
[1] "This"     "Is"       "A"        "String"
[5] "Variable"
```

There are many functions that work with character variables.

```
y <- c("This", "Is", "A", "String", "Variable")
grep("A", y, ignore.case = T)
```

```
[1] 3 5
```

This `grep()` function tells you which element(s) of a vector contain a pattern that you give it.

A Note on Assignment

When we created the variable `x`, we used an *assignment operator*. There are two primary assignment operators in **R**:

1. The arrow: `<-`
2. The equal sign: `=`

Either one works. `x <- c(1, 5, 3, 6)` and `x = c(1, 5, 3, 6)` will both define a new vector `x` the same way.

Note: There is a somewhat fierce debate about assignment operators in **R** with some arguing the arrow is the only way it should be done to avoid confusion in functions (and for tidiness and tradition). Others argue that the arrow is outdated since it takes more key strokes and relies too much on white space since `x <- 5` and `x < -5` are different commands. The first assigns the value 5 to `x` and the second checks if `x` is less than -5.

R Libraries

Often times, **R** will not have a built-in function that does what you want to do. In that case, you can usually find a package (or library) you can install that will have other functions. You can install **R** packages by issuing the `install.packages()` function. You will need to put the name of the package in quotes. For example, to install the `knitr` package, you can type

```
install.packages("knitr")
```

It may prompt you to choose an install source. You can always select the Cloud for this. Once it is installed, you do not need to install it again. You will, however, need to load it each time you reopen **R**. You can load a package with the `library()` function. The name of the package does not need to be in quotes anymore, but it can be.

```
library(knitr)
```


R Libraries

The `knitr` library has many functions that base **R** does not. For example, the `kable()` function in the `knitr` packages will convert a data frame to a nice table

```
kable(mtcars[1:5, 1:5], format = "latex")
```

	mpg	cyl	disp	hp	drat
Mazda RX4	21.0	6	160	110	3.90
Mazda RX4 Wag	21.0	6	160	110	3.90
Datsun 710	22.8	4	108	93	3.85
Hornet 4 Drive	21.4	6	258	110	3.08
Hornet Sportabout	18.7	8	360	175	3.15

There are many different packages that we will be installing and using this semester.

Section 2

Data Structures

Introduction

A data structure is a particular way of organizing data in a computer so that it can be used effectively. The idea is to reduce the space and time complexities of different tasks. Data structures in **R** programming are tools for holding multiple values.

R's base data structures are often organized by their dimensionality (1D, 2D, or nD) and whether they're homogeneous (all elements must be of the identical type) or heterogeneous (the elements are often of various types). This gives rise to the six data types which are most frequently utilized in data analysis.

Data Structure Types

The most essential data structures used in **R** include:

- Vectors
- Data Frames
- Factors

Vectors

A vector is an ordered collection of basic data types of a given length. The only key thing here is all the elements of a vector must be of the identical data type e.g homogeneous data structures. Vectors are one-dimensional data structures.

We can create a vector in **R** using the `c()` function. Separate values with commas. Note: other functions, like the `rep()` function, can be used inside the `c()` function.

```
x <- c(10, 12, 3, 4, 5, 5) # Define vector x
x <- c(10, 11 + cos(0), 3:4, rep(5, 2)) # Operations work in t
x # Print vector x to screen. print(x) also works
```

```
[1] 10 12  3  4  5  5
```


Functions with Vectors

R makes it very easy to work with vectors. If you put a vector in a function that works on a single number, it will apply that function to every element of the vector.

```
x = c(10, 12, 3, 4, 5, 5) # Define x differently
rep(x, 2) # Repeat x twice
```

```
 [1] 10 12  3  4  5  5 10 12  3  4  5  5
```

Many functions expect a vector to be entered.

```
length(x) # Report how many elements are in x
```

```
[1] 6
```

```
        # Takes the difference between
diff(x) # values in x that are one apart
```

```
[1]  2 -9  1  1  0
```


Operations with Vectors

If we perform an arithmetic operation of a number with a vector, then it will do the operation element-wise (to each element in the vector).

```
x + 2 # Add 2 to every element in x
```

```
[1] 12 14  5  6  7  7
```

```
2*x # Multiply each element in x by 2
```

```
[1] 20 24  6  8 10 10
```

```
x^2 # Square each element in x
```

```
[1] 100 144   9  16  25  25
```

```
x > 3 # Check whether each element is greater than 3
```

```
[1]  TRUE  TRUE FALSE  TRUE  TRUE  TRUE
```


Subsetting Vectors

If we want to work with only certain elements in a vector, we can subset it using square brackets `[]`.

```
x[1:2] # Takes the first two elements in x
```

```
[1] 10 12
```

```
x[c(1,2)] # Another way to do that
```

```
[1] 10 12
```

We can exclude certain elements in x by using negative values in the brackets.

```
x[-c(2,5)] # Excludes the 2nd and 5th elements of x
```

```
[1] 10  3  4  5
```


Subsetting Vectors

We can also put a logical statement in the `[]` when subsetting.

```
x
```

```
[1] 10 12  3  4  5  5
```

```
x[x > 5]
```

```
[1] 10 12
```

```
x[x == 5 | x == 10] # The | means "or"
```

```
[1] 10  5  5
```

```
x[x > 5 & x < 11]   # The & means "and"
```

```
[1] 10
```


Working with Multiple Vectors

We can also add, subtract, multiply, or divide one vector by another as long as the sizes are the same. These operations will be performed pairwise.

```
y = c(10, 5, 3, 2, 7, 9)
x + y
```

```
[1] 20 17  6  6 12 14
```

This next line will give an unexpected result since the sizes are not the same.

```
x + y[1:5]
```

```
[1] 20 17  6  6 12 15
```

Data Frames

Data frames are generic data objects of **R** which are used to store the tabular data. Data frames are the foremost popular data objects in **R** programming because we are comfortable in seeing the data within the tabular form. They are two-dimensional, heterogeneous data structures. These are lists of vectors of equal lengths.

Data frames have the following constraints placed upon them:

- A data-frame must have column names and every row should have a unique name. If you don't specify the row name, **R** will give it a number by default.
- Each column must have the identical number of items.
- Each item in a single column must be of the same data type.
- Different columns may have different data types.
- To create a data frame we use the `data.frame()` function.

Data Frame Example

Let's load a data frame that is already in **R** using the `data()` function. We can check the object we load into memory is a data frame by using the `class()` function.

```
data(USArrests)
class(USArrests)
```

```
[1] "data.frame"
```

We can observe the first rows of a data frame using the `head()` function. By default, `head()` will dispaly the first 6 rows, but we can adjust that.

```
head(USArrests, 3) # Displays the first three rows.
```

```
        Murder Assault UrbanPop Rape
Alabama   13.2     236       58 21.2
Alaska    10.0     263       48 44.5
Arizona    8.1     294       80 31.0
```


Data Frame Example

This USArrests data frame has 4 columns (variables) and 50 rows. We can check that using the `ncol()` and `nrow()` functions or by using the `dim()` function.

```
ncol(USArrests); nrow(USArrests); dim(USArrests)
```

```
[1] 4
```

```
[1] 50
```

```
[1] 50  4
```

This data frame's columns are named Murder, Assault, UrbanPop, and Rape. The UrbanPop variable gives the percentage of the state's population that lives in urban areas. The other variables give the number of arrests for that crime per 100,000 people.

There is no state variable. Instead, the rows are named after the states. The command `rownames(USArrests)` will give the names of each row.

Subsetting Data Frames

VERY IMPORTANT

We can access a specific column of the data frame using the $ operator and then typing the name of the variable (or column) that we are interested in.

```
USArrests$Murder
```

```
 [1] 13.2 10.0  8.1  8.8  9.0  7.9  3.3  5.9 15.4 17.4
[11]  5.3  2.6 10.4  7.2  2.2  6.0  9.7 15.4  2.1 11.3
[21]  4.4 12.1  2.7 16.1  9.0  6.0  4.3 12.2  2.1  7.4
[31] 11.4 11.1 13.0  0.8  7.3  6.6  4.9  6.3  3.4 14.4
[41]  3.8 13.2 12.7  3.2  2.2  8.5  4.0  5.7  2.6  6.8
```

We can achieve the same thing by using square brackets after the name of the data frame. Since the murder values are in the first column, we can type

```
USArrests[,1] # Will also give the Murder values
```


Subsetting Data Frames

If we only want specific rows, we can put those before the comma in the square bracket

```
USArrests[c(1,5),] # Gives the 1st and 5th rows
```

```
           Murder Assault UrbanPop Rape
Alabama      13.2     236       58 21.2
California    9.0     276       91 40.6
```

We can also select a specific value of a specific column by putting the row number first and the column number second. Or we can use the $ operator.

```
USArrests[10, 2] # Gives Assault rate for the 10th state
```

```
[1] 211
```

```
USArrests$Assault[10] # Another way to get the same value
```

```
[1] 211
```


Creating Data Frames

We can create a data frame using the `data.frame()` function. It is best to give a name to each column and then list what we want for each column.

```
df1 <- data.frame(Name=c("Juan", "Carlos"), Age = c(22, 31))
df1
```

```
    Name Age
1   Juan  22
2 Carlos  31
```

We can add another column to the data frame using the `cbind()` function.

```
vote <- c(TRUE, FALSE) # Create a new vector
df2 <- cbind(df1, vote)
df2
```

```
    Name Age  vote
1   Juan  22  TRUE
2 Carlos  31 FALSE
```

Changing Variable Names

You can also use the `rbind()` function to add more rows.

We can change the name of a variable using the `names()` function. We can change the names of the rows too. Suppose I want to change all the variable names to be lowercase and add row names. I can type the following.

```
names(df2) <- c("name", "age", "vote")    # Change column names
rownames(df2) <- c("person1", "person2") # Change row names
df2
```

```
          name age  vote
person1   Juan  22  TRUE
person2 Carlos  31 FALSE
```


Factors

Factors are the data objects which are used to categorize the data and store it as levels. They are useful for storing categorical data. They can store both strings and integers. They are useful to categorize unique values in columns like "TRUE" or "FALSE", or "MALE" or "FEMALE", etc.. They are useful in data analysis for statistical modeling.

To create a factor in **R** you need to use the function called `factor()`. The argument to this `factor()` is the vector.

```
fac = factor(c("Male", "Female", "Male",
               "Male", "Female", "Male", "Female"))
fac
```

```
[1] Male   Female Male   Male   Female Male   Female
Levels: Female Male
```


Working with Factors

We can rearrange the levels of the factor (which are alphabetical by default) also using the `factor()` function.

```
fac = factor(fac, levels = c("Male", "Female"))
fac
```

```
[1] Male   Female Male   Male   Female Male   Female
Levels: Male Female
```

We can also force a variable to be a factor using the `as.factor()` function.

```
department = c("Math", "CS", "Other")
class(department)
```

```
[1] "character"
```

```
dep_factor = as.factor(department)
class(dep_factor)
```

```
[1] "factor"
```


Section 3

R Functions

R Functions

We have already used functions in **R**, but this topic will expand our knowledge of functions and introduce writing our own functions.

First, consider the function `seq()`. If we look at the documentation of the sequence function (by typieng `?seq`), it will tell us the syntax is

```
seq(from = 1, to = 1, by = ((to - from)/(length.out - 1)),
length.out = NULL, along.with = NULL, ...)
```

So, if we type `seq(1, 5, 2)`, then it will output the numbers from 1 to 5 counting by 2s since those are the first three arguments.

```
seq(1, 5, 2)
```

```
[1] 1 3 5
```


Working with Functions

If I want my third number to instead be the length of the vector, then I need to specify that.

```
seq(1, 5, length.out = 2)
```

```
[1] 1 5
```

In fact, it is not required to type the entire name of that `legnth.out` argument. You just need to type enough letters for **R** to know what argument you are referring to.

```
seq(1, 5, length = 2)
```

```
[1] 1 5
```

```
seq(1, 5, l = 2)
```

```
[1] 1 5
```

Even just typing `l` was enough since no other argument starts with that letter. It is good practice, however, to type enough for clarity.

Default Function Inputs

Additionally, many functions will have a default entry for an argument. For example, the `seq()` function, by default, has a lower and upper bound of 1. So, if you just type

```
seq()
```

```
[1] 1
```

It gives an output.

Another example: the `sample()` function randomly selects values of an input vector. The syntax is

```
sample(x, size, replace = FALSE, prob = NULL)
```

So, by default it will sample without replacement. Typing `replace = T` or `replace = TRUE` will have it sample with replacement.

Note: we should always put an equal sign = not an arrow <- when telling **R** what our function inputs are in the function.

Writing Functions

It is possible to write your own function in **R**. Writing a function begins with the `function()` function. The syntax is

```
function_name <- function(all inputs) { function_body }
```

Example: if we want to write a function that takes an input, divides it by 2, and then squares it, we will type

```
halfsquare <- function(x) {
  result <- (x/2)^2
  return(result)
}
halfsquare(10)
```

```
[1] 25
```

In this function, we specified that x was the input, so we can refer to x in the function even if x was never defined globally.

Writing Functions

It is important to make sure the variable we want the function to output is listed on the last line of the function body. Some will argue that should be put in a `return()` function, but it doesn't matter.

We can also give our function default inputs. When defining the function, just let the input equal something. That way, it will still return something even when no input is given.

```
halfsquare <- function(x = 2) {
  result <- (x/2)^2
  result
}
halfsquare(10)
```

```
[1] 25
```

```
halfsquare()
```

```
[1] 1
```


Writing Functions

The functions we write can have any number of inputs.

```
diff_func <- function(x, y, z = 0) {
  d <- x - y - z
  d
}
diff_func(100, 10, 1)
```

```
[1] 89
```

When writing your own functions, make sure you do not name it anything that is already a built-in **R** function. For example, there is already a `diff()` function, which is why I named this `diff_func`. This is good practice for all variable names as well.

Documenting Functions

When we write a function that is a bit complicated, it is good practice to document the function to explain how it works. This can be done in comments in the function body.

```
diff_func <- function(x, y, z = 0) {
  # Takes the first entered value and subtracts the others.
  # A third number is optional.
  d <- x - y - z
  d
}
```


Function Code

If you are curious about the code used to define a function, you can simply type the function name without parentheses.

```
diff_func
```

```
function (x, y, z = 0)
{
    d <- x - y - z
    d
}
```

Function Code

Many built-in functions, however, simply call a version of the function written more efficiently in the C programming language. Therefore, we cannot see the exact code of the function. It will tell you what environment, or package, the function is in.

```
rnorm
```

```
function (n, mean = 0, sd = 1)
.Call(C_rnorm, n, mean, sd)
<bytecode: 0x1176cd870>
<environment: namespace:stats>
```


Functions with Same Name

Sometime you will encounter different functions that have the same name. This can occur when you load different **R** packages that each have a function with the same name or when you name your function a name that already exists. **R** can know which function you want to use if you type the name of the packages followed by two colons (`::`) with no spaces between them.

So, the syntax is

```
package_name::function_name()
```


Functions with Same Name

For example, suppose I wrote function to calculate the weighted average of a vector and called that function `mean`.

```
mean <- function(x, weight) {
  sum(x * weight) / sum(weight)
}
x <- c(1, 2, 3, 4, 5)
w <- c(5, 1, 5, 1, 3) # Vector of weights
mean(x, w)
```

```
[1] 2.733333
```

Of course, there already is a built-in `mean` function that now we cannot use by typing `mean(x)`, since it will only use the new function I wrote. To use the built-in `mean()` function, which is in the base environment, I can type

```
base::mean(x) # The syntax is package::function()
```

```
[1] 3
```


Removing Functions

To remove any object that you create, you can use the `rm()` function. In this case, it would be better if I remove the function called `mean()` that I wrote and name it something else so there is no confusion. It is not possible to remove a function that is built-in or loaded in with an **R** package.

```
rm(mean)
weighted_mean <- function(x, weight) {
  sum(x * weight) / sum(weight)
}
x <- c(1, 2, 3, 4, 5)
w <- c(5, 1, 5, 1, 3) # Vector of weights
weighted_mean(x, w)
```

```
[1] 2.733333
```

```
mean(x) # The unweighted, built-in mean function
```

```
[1] 3
```


Section 4

Data Import and Export

Introduction

When working with **R** it is not typical that we will use data sets that are loaded with packages or that we type the data in ourselves. We will often want to read in data files that are stored on our computers.

We can read in data files using a couple **R** functions. But first, it is important to know common ways that data sets are stored.

Types of Data Files

There a couple very common types of data files

- `.txt` files. These are the standard text file.
- `.csv` files. These are comma separated value files. Microsoft Excel can save files this way.
- `.xlsx` files. These are Microsoft Excel files.
- `.xls` files. These are an older type of Microsoft Excel file.
- `.json` files. These are files that have data stored in key-value pairs and arrays
- `.dat` files. These are another type of file that store data.

In this class, we will primarily be using `.txt`, `.csv`, and `.xlsx` files. So we will focus on those.

How Data Are Stored

In a data file, we will usually want the data stored in **tidy format**. This indicates that the data file is set up similar to a data frame with one observation per row. Then each row can have the information from all of the variables.

However, it is important to know how the data values are separated from each other. One way to do this is with a space. These are called **space delimited files**. In this case, the data may look like this:

```
Age Weight
31 220
15 94
52 131
```

This is also an example of a data file that has a header. That is, the first row contains the column names, not data values.

How Data Are Stored

Another way is to use a comma. If commas are used, it is called a **comma separated value** or **comma delimited** file (even if the file name does not end in `.csv`).

```
Age,Weight
31,220
15, 94
52,131
```

Spaces in these files are ignored unless they are in quotes.

There are other ways to separate data values like using tab delimiters or new line delimiters. Comma and space are the most common, though.

Reading in Data Files

There are two primary functions we can use to read in data files.

- `read.csv()`
- `read.table()`

The `read.csv()` function is simpler, but not as powerful or adaptable as the `read.table()` function.

The `read.csv()` Function

This function works best for csv files or text files with comma deliminators. The syntax is

```
read.csv(file, header = TRUE, sep = ",", quote = "\"",
         dec = ".", fill = TRUE, comment.char = "", ...)
```


The `read.table()` Function

This is the more complicated, but more customizable, function. The syntax is given below, but many of these arguments will not be used.

```
read.table(file, header = FALSE, sep = "", quote = "\"'",
           dec = ".", numerals = c("allow.loss",
                                   "warn.loss", "no.loss"),
           row.names, col.names, as.is = !stringsAsFactors,
           tryLogical = TRUE,
           na.strings = "NA", colClasses = NA, nrows = -1,
           skip = 0, check.names = TRUE,
           fill = !blank.lines.skip,
           strip.white = FALSE, blank.lines.skip = TRUE,
           comment.char = "#",
           allowEscapes = FALSE, flush = FALSE,
           stringsAsFactors = FALSE, fileEncoding = "",
           encoding = "unknown", text, skipNul = FALSE)
```

Using the `read.csv()` and `read.table()` Functions

To use these functions, we need to let **R** know exactly where the file is. We can do this in one of two ways.

1. Set the working directory as the folder that contains the file.
2. Put the file path in the `read.csv()` or `read.table()` function we are using.

Setting the Working Directory

Here are the steps to set a new working directory in **RStudio**.

1. Click on "Session" in the menu bar.
2. Go down to "Set Working Directory" and then "Choose Directory".
3. Navigate to the folder (not file) that contains the file you want to read in.
4. Click "Open".

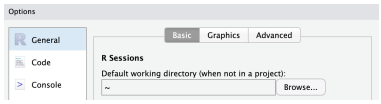
Now the directory will be set and you can put the file name into the `read.csv()` or `read.table()` function.

Note: you will need to set the working directory every time you quit and reopen **RStudio**. You can avoid this by changing the default directory to a folder than will contain all of your data files for this class if you'd like.

Set the Default Directory

The steps for changing the default directory are:

1. Open the **RStudio** settings by clicking on "Tools" from the menu bar and the going down to "Global Options".
2. The settings will open (that look like the image below). Click on the Browse... button under the option for default directory.
3. Navigate to the folder (not file) that contains the file you want to read in and click "Open".

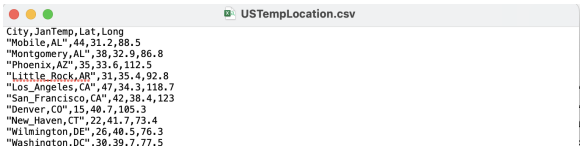


4. Click "Apply" at the bottom right side of the settings pane to save and close the settings.

Read Data File using `read.csv()`

Once we have changed our directory, we are ready to read in a file. Suppose I want to read in the file called `USTempLocation.csv`.

Here is what the file looks like if we open it in a text editor.



Read Data File using `read.csv()`

Since this is a csv file, we will begin using the `read.csv()` function.

```
dat <- read.csv("USTempLocation.csv")
head(dat)
```

Notice that by default, the `read.csv()` function will use the first row in the file as column names. We can change this by using the `header=F` argument, but we usually want the first row to be names.

Read Data File using `read.table()`

We can also use the `read.table()` function to read in that data file, but a few more options need to be added.

```
dat <- read.table("USTempLocation.csv", sep = ",",
                  header=T)
head(dat)
```

```
               City JanTemp  Lat  Long
1        Mobile,AL      44 31.2  88.5
2    Montgomery,AL      38 32.9  86.8
3       Phoenix,AZ      35 33.6 112.5
4   Little_Rock,AR      31 35.4  92.8
5   Los_Angeles,CA      47 34.3 118.7
6 San_Francisco,CA      42 38.4 123.0
```

Read Data File using `read.table()`

By default, `read.table()` will use a space deliminator, not a comma deliminator, and it will not count the first row as column names. If we had used the default options, then it would have looked like this:

```
dat <- read.table("USTempLocation.csv")
head(dat)
```

```
                  City.JanTemp.Lat.Long
Mobile,AL                  ,44,31.2,88.5
Montgomery,AL              ,38,32.9,86.8
Phoenix,AZ                ,35,33.6,112.5
Little_Rock,AR             ,31,35.4,92.8
Los_Angeles,CA            ,47,34.3,118.7
San_Francisco,CA            ,42,38.4,123
```

Not good!

Read Data File using `read.table()`

The `read.table()` function is also useful when the data file is not comma delimited. For instance, suppose I want to read the file `turkey.txt`, which contains info on the bone lengths of turkey and whether they are wild or not. The data file looks like this:

```
id humerus tibiotarsus type
B093 151 152 0
B102 153 156 0
B103 149 151 0
L641 150 141 0
L783 154 157 0
L905 169 163 0
B710 153 151 0
B790 156 155 0
B819 158 152 0
```

By default, `read.table()` will assume the data is space delimited, like it is in this case.

So we can read it in using this code:

```
turkey <- read.table("turkey.txt", header = T)
head(turkey, 2)
```

```
    id humerus tibiotarsus type
1 B093     151          152    0
2 B102     153          156    0
```


Read Data File Without Setting Directory

We can also read a data file without setting the directory to be where the file is beforehand as long as we put the path to the file in the function we are using.

For example, if my data file is on the desktop of my computer, then the file path would be "~/Desktop/" and then I would put the file name after that.

For Mac users, the "~" character represents the home folder of the logged-in user. You can determine the path to a file by clicking on it and typing "option-command-c".

On Windows, if you click the Start button and then click Computer, click to open the location of the desired file, hold down the Shift key and right-click the file. Then on that menu, you can select "Copy as path".

```
dat <- read.table("~/Desktop/USTempLocation.csv")
head(dat)
```


Reading Excel Files

In the `readxl` package is the function `read_excel()`, which can read in files that end in `.xls` or `.xlsx`.

```
install.packages("readxl")
library(readxl)
dat <- read_excel("~/Desktop/USTempLocation.xlsx")
head(dat)
```

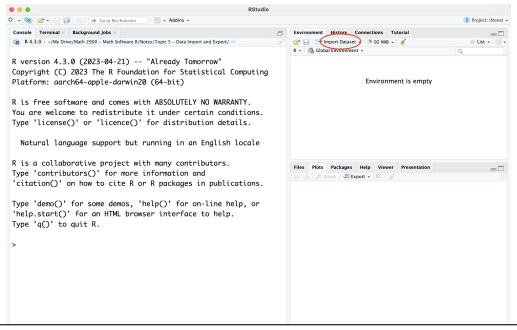
If there is more than one sheet in the Excel notebook, you can specify which one using the `sheet` option in the `read.excel` function.

Note: the `read_excel()` function will read in the data file as a *tibble*, not as a data frame. We will discuss tibbles more in Section 5. Tibbles work just like data frames for nearly all practical things we do with them.

Reading Data Files Using **RStudio** Import Dataset

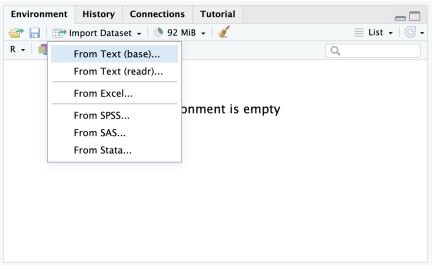
While I recommend using the `read.csv()` or `read.table()` functions becuase then it is easy to run the code again to read in the file, you can also read in data files using the "Import Dataset" button in **RStudio**.

In the Environment tab on the top right, there is an Import Dataset button.



Reading Data Files Using **RStudio** Import Dataset

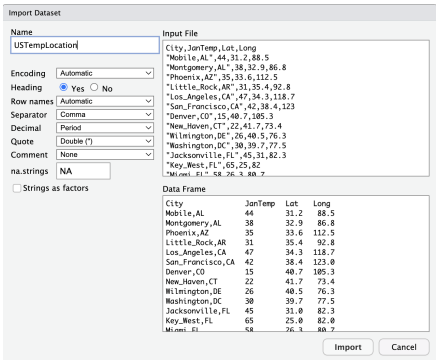
Clicking on the Import Dataset button will give these options:



Click on "From Text (base)...", navigate to the data file on your computer, and then click "Open".

Reading Data Files Using **RStudio** Import Dataset

That will bring up this window. We then can change the options as needed.



Exporting Data

We can export a data frame into a file using the `write.csv()` or `write.table()` function. The syntax of `write.table()` is

```
write.table(x, file = "", append = FALSE, quote = TRUE,
            sep = " ", eol = "\n", na = "NA", dec = ".",
            row.names = TRUE, col.names = TRUE,
            qmethod = c("escape", "double"),
            fileEncoding = "")
```

`x` is the data frame we want to export and then we need to put the file name (and a file path if we don't want it saved in our working directory)

Like the read functions, `write.csv()` is just a simpler version of `write.table()`.

Exporting Data

We can make a data frame and then export it as a text file using the following code

```
Xdf = data.frame(age = c(10, 24, 51, 14),
                 state = c("UT", "UT", "CA", "CA"))
Xdf
```

```
  age state
1  10    UT
2  24    UT
3  51    CA
4  14    CA
```

```
write.table(Xdf, "age_state.txt", sep = ",")
```

We won't need to do this often, but it is good to be able to save data frames easily into data files.

Section 5

Tidyverse

Introduction

Modern **R** users are migrating away from many base **R** packages and functions to instead work in the **tidyverse**.

The tidyverse is both a philosophy for coding and organizing data as well as a collection of packages in **R**.

```
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────── tidyverse 2.0.0 ──
✔ dplyr     1.1.4     ✔ readr     2.1.4
✔ forcats   1.0.0     ✔ stringr   1.5.1
✔ ggplot2   3.4.4     ✔ tibble    3.2.1
✔ lubridate 1.9.3     ✔ tidyr     1.3.0
✔ purrr     1.0.2
── Conflicts ────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package to force all conflicts to become errors
```


Tidy Format (murders data)

We say that a data table is in **tidy** format if each row represents one observation and columns represent the different variables available for each of these observations. For example, the following data set is in tidy format:

```
library(dslabs)
data(murders)
head(murders)
```

```
       state abb region population total
1    Alabama  AL  South    4779736   135
2     Alaska  AK   West     710231    19
3    Arizona  AZ   West    6392017   232
4   Arkansas  AR  South    2915918    93
5 California  CA   West   37253956  1257
6   Colorado  CO   West    5029196    65
```

Not Tidy Format (fertility)

The following dataset is organized, but not tidy. Why?

```
##       country 1960 1961 1962
## 1     Germany 2.41 2.44 2.47
## 2 South Korea 6.16 5.99 5.79
```


Tidy Format (fertility)

Here is what the data would look like in tidy format:

```
      country year fertility
1     Germany 1960      2.41
2 South Korea 1960      6.16
3     Germany 1961      2.44
4 South Korea 1961      5.99
5     Germany 1962      2.47
6 South Korea 1962      5.79
```

The same information is provided, but there are important differences in the format. For the **tidyverse** packages to be optimally used, data need to be reshaped into 'tidy' format. The advantage of working in tidy format allows the data analyst to focus on more important aspects of the analysis rather than the format of the data.

Tibbles

A **tibble** is a modern version of a data.frame.

```
library(tidyverse)
dat1 <- tibble(x = 1:4, y = 5:8, z = c("A", "B", "C", "D"))
```

Or convert a data frame to a tibble

```
dat <- data.frame(x=1:4, y = 5:8, z = c("A", "B", "C", "D"))
dat1 <- as_tibble(dat)
dat1
```

```
# A tibble: 4 x 3
      x     y z
  <int> <int> <chr>
1     1     5 A
2     2     6 B
3     3     7 C
4     4     8 D
```


Tibbles

Important characteristics that make tibbles unique:

1. Tibbles are primary data structure for the `tidyverse`
2. Tibbles display better and printing is more readable
3. Tibbles can be grouped
4. Subsets of tibbles are tibbles
5. Tibbles can have complex entries–numbers, strings, logicals, lists, functions, other tibbles, etc.

Subsetting Tibbles

Note: tibbles work just like data frames in just about every way except one. With data frames, using brackets [] will give vectors and with tibbles, using [] will give tibbles.

```
class(dat[,1])  # Class of first column from a data frame
```

```
[1] "integer"
```

```
class(dat1[,1]) # Class of first column from a tibble
```

```
[1] "tbl_df"     "tbl"        "data.frame"
```

```
mean(dat[,1])
```

```
[1] 2.5
```

```
mean(dat1[,1])
```

```
[1] NA
```


Subsetting Tibbles

We can choose the columns of a tibble as a vector using the $ operator or by putting double brackets [[]].

```
dat1$x    # Gives the first column (whose name is x)
```

```
[1] 1 2 3 4
```

```
dat1[[1]] # Gives the first column
```

```
[1] 1 2 3 4
```

```
mean(dat1[[1]])
```

```
[1] 2.5
```

This subsetting is not a problem for rows. Rows of data frames are data frames and rows of tibbles are tibbles, so nothing of note changes there.

Data Import in the Tidyverse

The `tidyverse` has its own functions that will read in data sets as tibbles. They are the functions

- `read_csv()`
- `read_table()`

These work very much like the `read.csv()` and `read.table()` functions. The primary difference is that `read.csv()` and `read.table()` read in the data has a data frame whereas `read_csv()` and `read_table()` read in the data as a tibble.

The `read_excel()` function in the `readxl` library also reads in data files as tibbles even though the `readxl` library is not technically part of the tidyverse.

`dplyr` Functions

One of the most useful packages in the `tidyverse` is the **dplyr** package that is used for data wrangling. `dplyr` is called that since it is a tool (like a set of pliers) for data frames (or tibbles).

The `dplyr` package has the following useful functions:

- `mutate()` adds new variables that are functions of existing variables.
- `filter()` picks cases based on their values. Selects rows.
- `select()` picks variables based on their names. Selects columns.
- `summarize()` or `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.
- `group_by()` allows you to perform any operation "by group"

Note an important point: most dplyr functions (and most functions in the tidyverse) input a tibble and then output a modified tibble, although many can also work with data frames.

Mutate

The function **mutate** takes the data frame or tibble, the instructions for the new columns in next arguments, and returns a modified data frame. For example:

```
murders <- as_tibble(murders)
head(murders)
```

```
# A tibble: 6 x 5
  state      abb   region population total
  <chr>      <chr> <fct>       <dbl> <dbl>
1 Alabama    AL    South     4779736   135
2 Alaska     AK    West       710231    19
3 Arizona    AZ    West      6392017   232
4 Arkansas   AR    South     2915918    93
5 California CA    West     37253956  1257
6 Colorado   CO    West      5029196    65
```


Mutate

To add murder rates, we mutate as follows:

```
murdersRate <- mutate(murders,
  rate = total / population * 100000
)
head(murdersRate)
```

```
# A tibble: 6 x 6
  state      abb   region population total  rate
  <chr>      <chr> <fct>       <dbl> <dbl> <dbl>
1 Alabama    AL    South     4779736   135  2.82
2 Alaska     AK    West       710231    19  2.68
3 Arizona    AZ    West      6392017   232  3.63
4 Arkansas   AR    South     2915918    93  3.19
5 California CA    West     37253956  1257  3.37
6 Colorado   CO    West      5029196    65  1.29
```


Filter

Now suppose that we want to filter the data table to only show the entries for which the murder rate is lower than 0.71. We do this as follows:

```
filter(murdersRate, rate <= 0.71)
murdersRate[murdersRate$rate <= 0.71,] # How do get the same
                                       # result without
                                       # filter function
```

```
# A tibble: 5 x 6
  state         abb   region        population total  rate
  <chr>         <chr> <fct>              <dbl> <dbl> <dbl>
1 Hawaii        HI    West             1360301     7 0.515
2 Iowa          IA    North Central    3046355    21 0.689
3 New Hampshire NH    Northeast        1316470     5 0.380
4 North Dakota  ND    North Central     672591     4 0.595
5 Vermont       VT    Northeast         625741     2 0.320
```


Select

If we want to view just a few of our columns, we can use the following:

```
murdersRate <- mutate(murders, rate = total / population * 100000)
murdersRateSelect <- select(murdersRate, state, rate)
filter(murdersRateSelect, rate <= 0.71)
# The above is the same as the following
murdersRate[murdersRate$rate <= 0.71, c("state", "rate")]
```

```
# A tibble: 5 x 2
  state          rate
  <chr>         <dbl>
1 Hawaii        0.515
2 Iowa          0.689
3 New Hampshire 0.380
4 North Dakota  0.595
5 Vermont       0.320
```

Nesting Functions

Instead of defining new objects along the way, we could do everything in one complex nested function:

```
filter(select(mutate(murders, rate = total / population * 100000),
              state, rate), rate <= 0.71)
```

```
# A tibble: 5 x 2
  state          rate
  <chr>         <dbl>
1 Hawaii        0.515
2 Iowa          0.689
3 New Hampshire 0.380
4 North Dakota  0.595
5 Vermont       0.320
```

This is fairly concise but a little confusing. Is there a better, clearer way?

Pipes

In the previous example, we performed the following wrangling operations:

$$\text{original data} \rightarrow \text{mutate} \rightarrow \text{select} \rightarrow \text{filter}$$

We can perform a series of operations in **R** by sending the results of one function to another using the **pipe operator**: |> that was added in **R** version 4.1.

There is also a pipe (that was actually added first) in the `magrittr` package that is loaded with the tidyverse with the syntax %>%. This `magrittr` pipe can do a few things the native pipe cannot[1], but for the vast majority of cases they work the same, so it is recommended to use the native pipe since it is built-in and runs slightly faster.

[1] https://magrittr.tidyverse.org

Pipes

The pipe is a combination of characters that when used properly does two things: *It shortens and simplifies the code* and it makes the code more intuitive to read.

There is a keyboard shortcut in RStudio for inserting the pipe. While you can always just type |>, you can also type:

Mac: Command-Shift-M
Windows: Control-Shift-M

However, this will not give you the default pipe unless you change a setting in RStudio. Go to
Tools → Global Options → Code → Select "Use native pipe operator".

Pipes

All the pipe does is provide **forward application** of an object to the first argument of a function. The pipe sends left side of the input to the function to the right of the pipe. For example, if we wanted to calculate

$$\log_2(\sqrt{16})$$

We could use:

```
16 |> sqrt() |> log2()
```

```
[1] 2
```

Since the pipe sends values to the first argument, we can define other arguments as follows:

```
16 |> sqrt() |> log(base = 2)
```

```
[1] 2
```

While piping works the way it is formatted above, it is better practice to use a new line after each pipe.

Pipes (murders)

Creating the prior tibble operation using pipes:

```
murders |>
  mutate(rate = total / population * 100000) |>
  select(state, rate) |>
  filter(rate <= 0.71)
```

```
# A tibble: 5 x 2
  state          rate
  <chr>         <dbl>
1 Hawaii        0.515
2 Iowa          0.689
3 New Hampshire 0.380
4 North Dakota  0.595
5 Vermont       0.320
```


Piping into Other Arguments

By default, pipes will send the object being piped to the first argument of the next command, but we can send it to another argument by using the underscore (_) placeholder and specifying the argument.

```
murdersRate |>
  lm(rate ~ population, data = _)
```

```
Call:
lm(formula = rate ~ population, data = murdersRate)

Coefficients:
(Intercept)   population
  2.575e+00    3.363e-08
```

We can also use the pipe placeholder along with $, [], and [[]]:

```
murdersRate |> _$rate |> head(5)
```

```
[1] 2.824424 2.675186 3.629527 3.189390 3.374138
```

Arrange

We know about the **order** and **sort** functions, but for ordering entire tables, the **arrange** function is much more useful. For example, here we order the tibble by the state's murder rate:

```
murdersRate |>
  arrange(rate) |>
  head()
```

```
# A tibble: 6 x 6
  state          abb   region        population total  rate
  <chr>          <chr> <fct>              <dbl> <dbl> <dbl>
1 Vermont        VT    Northeast         625741     2 0.320
2 New Hampshire  NH    Northeast        1316470     5 0.380
3 Hawaii         HI    West             1360301     7 0.515
4 North Dakota   ND    North Central     672591     4 0.595
5 Iowa           IA    North Central    3046355    21 0.689
6 Idaho          ID    West             1567582    12 0.766
```


Arrange (descending order)

Note that the default behavior is to order in ascending order. The function **desc** transforms a vector so that it is in descending order. To sort the table in descending order, we can type:

```
murdersRate |>
  arrange(desc(rate)) |>
  head()
```

```
# A tibble: 6 x 6
  state                abb   region        population total  rate
  <chr>                <chr> <fct>              <dbl> <dbl> <dbl>
1 District of Columbia DC    South             601723    99 16.5
2 Louisiana            LA    South            4533372   351  7.74
3 Missouri             MO    North Central    5988927   321  5.36
4 Maryland             MD    South            5773552   293  5.07
5 South Carolina       SC    South            4625364   207  4.48
6 Delaware             DE    South             897934    38  4.23
```


Nested sorting

If we are ordering by a column with ties, we can use a second (or third) column to break the tie. for example:

```
murdersRate |>
  arrange(region, rate) |>
  head()
```

```
# A tibble: 6 x 6
  state          abb   region     population total  rate
  <chr>          <chr> <fct>           <dbl> <dbl> <dbl>
1 Vermont        VT    Northeast      625741     2 0.320
2 New Hampshire  NH    Northeast     1316470     5 0.380
3 Maine          ME    Northeast     1328361    11 0.828
4 Rhode Island   RI    Northeast     1052567    16 1.52
5 Massachusetts  MA    Northeast     6547629   118 1.80
6 New York       NY    Northeast    19378102   517 2.67
```


Summarize

The **summarize** function computes summary statistics in an intuitive way. The 'heights' dataset includes heights and sex reported by students in an in-class survey.

```
data(heights)
heights |>
  filter(sex == "Female") |>
  summarize(
    avg = mean(height),
    std_dev = sd(height)
  )
```

```
       avg  std_dev
1 64.93942 3.760656
```


Group then summarize with group_by()

A common operation in data exploration is to first split data into groups and then compute summaries for each group. For example, we may want to compute the average and standard deviation for men's and women's heights separately. We can do the following

```
heights |>
  group_by(sex) |>
  summarize(
    average = mean(height),
    standard_deviation = sd(height)
  )
```

```
# A tibble: 2 x 3
  sex    average standard_deviation
  <fct>    <dbl>              <dbl>
1 Female    64.9               3.76
2 Male      69.3               3.61
```


pivot_longer() Function

Sometimes it is the case that the data need to be manually put into tidy format. This is where the `pivot_longer()` function can help.

```
prices <- read.csv("data/houseprice.txt")
head(prices, 10)
```

```
   gainesville orlando tampa
1        173.0   243.9 230.7
2        145.5   201.1 115.7
3        190.6   185.3 211.0
4        186.3   187.5 203.5
5        248.7   207.9 149.9
6        206.4   234.8 166.8
7         86.8   253.2 134.1
8        204.6   144.7 214.2
9        174.5      NA 105.5
10       220.0      NA 216.2
```

pivot_longer() Function

```
new_prices <- pivot_longer(prices, cols = everything())
head(new_prices)
```

```
# A tibble: 6 x 2
  name        value
  <chr>       <dbl>
1 gainesville  173
2 orlando      244.
3 tampa        231.
4 gainesville  146.
5 orlando      201.
6 tampa        116.
```

This looks much better! Except the column names are just set to the default of `name` and `value`. Also, there are some NA values as we saw in the original data set.

pivot_longer() Function

```
house_prices <- pivot_longer(prices, cols = everything()) |>
  na.omit() |>
  rename(city = name, price = value) |>
  arrange(city)
head(house_prices)
```

```
# A tibble: 6 x 2
  city        price
  <chr>       <dbl>
1 gainesville  173
2 gainesville  146.
3 gainesville  191.
4 gainesville  186.
5 gainesville  249.
6 gainesville  206.
```


pivot_wider() Function

It's relatively rare to need `pivot_wider()` to make tidy data, but it can be useful for creating summary tables for presentation, or data in a format needed by other tools. We won't focus too much on `pivot_wider()`, but here is an example. We do need an "id" or a "row number" variable to make this work.

```
price_wide <- house_prices |>
  mutate(row_num = c(1:11, 1:8, 1:10)) |>
  pivot_wider(names_from = city, values_from = price)
head(price_wide, 3)
```

```
# A tibble: 3 x 4
  row_num gainesville orlando tampa
    <int>       <dbl>   <dbl> <dbl>
1       1         173     244.  231.
2       2         146.    201.  116.
3       3         191.    185.  211
```


More on the tidyverse

There are some other tidyverse operations, including the `inner_join()`, `left_join()`, `right_join()`, `full_join()`, `pull()`, `dot()`, `reframe()`, `nest_by()`, and `pick()` functions. We will work with a few of these throughout the course.

Section 6

Graphing in Base **R** (not a focus for us)

Introduction

One of the primary reasons **R** became so popular is its graphics capabilities. In this topic, we discuss how to create and customize plots.

R has many built-in functions for graph creation:

- `plot()`: used for making scatter plots for two quantitative variables.
- `hist()`: used for making histograms for a quantitative variable.
- `boxplot()`: used for making box plots for a quantitative variable.
- `barplot()`: used for making bar plots for a categorical variable.
- `pie()`: used for making pie charts for a categorical variable (not recommended).

There are many other plots, but we will start here.

The plot() Function

The plot() function is probably the most versatile and complicated of all the functions listed on the previous slide. The syntax is

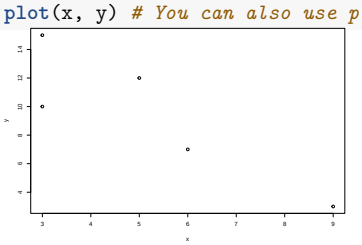
```
plot(x, y = NULL, type = "p", xlim = NULL, ylim = NULL,
     log = "", main = NULL, sub = NULL, xlab = NULL,
     ylab = NULL, ann = par("ann"), axes = TRUE,
     frame.plot = axes, panel.first = NULL,
     panel.last = NULL, asp = NA,
     xgap.axis = NA, ygap.axis = NA,
     ...)
```

There are many arguments we can use to adjust and customize the output of the plot.

The plot() Function

At its most basic, plotting in **R** is very easy.

```
x <- c(3, 3, 5, 6, 9)
y <- c(15, 10, 12, 7, 3)
plot(x, y) # You can also use plot(y~x)
```



This plot, however, is very boring and possibly difficult to read.

Customizing Plots

We can customize plots by using the arguments of the plot function. Here are some commonly arguments and what they do:

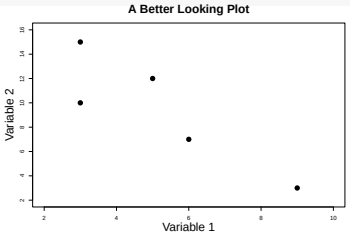
- type: Changes the type of graph. By default, the graph will put points for each (x,y) pair. This can be changed to "l" for lines, "b" for both lines and points, "h" for vertical lines up to the height of the point, "n" for no output, etc.
- xlim: Adjusts the bounds on the x axis. Requires a vector c(lower, upper).
- ylim: Adjusts the bounds on the y axis. Requires a vector c(lower, upper).
- xlab: Gives the label on the x axis. Requires a string in quotes
- ylab: Gives the label on the y axis. Requires a string in quotes.
- main: Gives the title of the plot. Requires a string in quotes.

Customizing Plots (Continued)

- cex: Adjusts the size of what is plotted. Requires a number. The default size is 1.
- cex.lab: Adjusts the font size of the axis labels. Requires a number. The default size is 1.
- cex.main: Adjusts the font size of the main title. Requires a number. The default size is 1.
- pch: Changes the look of what is plotted. Requires a number or a symbol in quotes.
- lty: This changes the type of the line that is plotted if type = "l". Requires a number. The default type is 1 (solid).
- lwd: This changes the width of the line that is plotted if type = "l". Requires a number. The default width is 1.
- col: Adjusts the color of the plot. Requires a number or a string with the color name in quotes.
- mgp: Adjusts where the labels, axis numbers, and axes are on the plot. Requires a vector of length 3. The default is c(3,1,0).

The plot() Function Customized

```
x <- c(3, 3, 5, 6, 9)
y <- c(15, 10, 12, 7, 3)
plot(x, y, xlab = "Variable 1", ylab = "Variable 2",
     main = "A Better Looking Plot", pch = 16, cex = 2,
     cex.lab = 2, cex.main = 2, mgp = c(2.5,1,0),
     xlim = c(2, 10), ylim = c(2, 16))
```

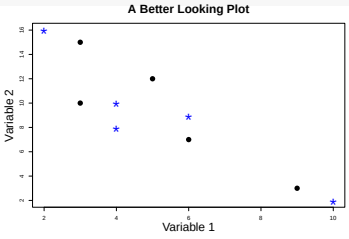


The points() and lines() Functions

If we ever want to add more points or additional lines to our graph, we can do so using the points() and lines() functions, respectively.

These functions do not create a new plot. They take the existing plot and simply add to it. For example, adding on to the previous graph.

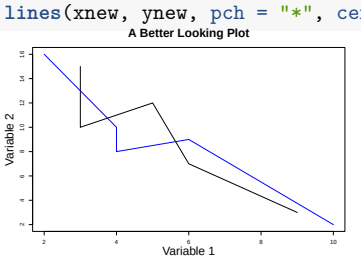
```
xnew <- c(2, 4, 4, 6, 10); ynew <- c(16, 10, 8, 9, 2)
points(xnew, ynew, pch = "*", cex = 3, col = "blue")
```



The points() and lines() Functions

If we had done a line plot and used the `lines()` function, it would look like this:

```
xnew <- c(2, 4, 4, 6, 10); ynew <- c(16, 10, 8, 9, 2)
lines(xnew, ynew, pch = "*", cex = 3, col = "blue")
```



Creating a Legend

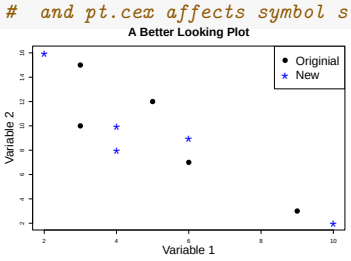
When making a more complicated plot with multiple lines or types of points, it is good practice to include a legend. We can do this with the `legend()` function.

There are lots of arguments for the legend function, but at its most basic, we should just match all the options we used when plotting. We can specify an exact position of the legend using `x` and `y` values, or you can just type a position like "topright" or "bottom". Then you need some text as the next argument. An example is on the next slide.

Creating a Legend Example

Again, adding to the last plot, we get:

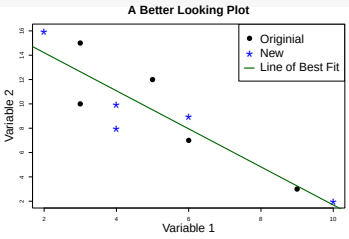
```
legend("topright", c("Originial", "New"), pch = c(16, 42),
       cex = c(2, 2), col = c("black", "blue"),
       pt.cex = c(2, 3))
# In the legend, cex affects text size
#  and pt.cex affects symbol size.
```



Adding a Line to the Plot

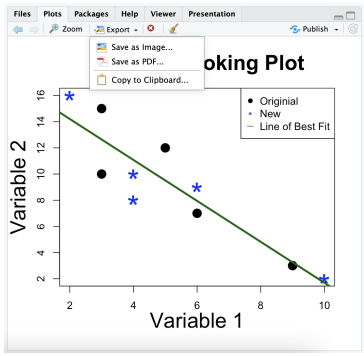
We can add a line of best fit through those data using the `abline()` function.

```
abline(17.3377, -1.5649, col = "darkgreen", lwd = 3)
legend("topright", c("Originial", "New", "Line of Best Fit"),
       pch=c(16, 42, 95), cex=c(2, 2, 2), pt.cex=c(2, 3, 3),
       col = c("black", "blue", "darkgreen"))
```



Saving Plots

You can save a plot by clicking on the Export button in the Plots viewer in **RStudio**.



Saving Plots

You can also save plots by using the `pdf()`, `png()`, or `jpeg()` functions. For any of those functions, we can use the following syntax (shown with the `pdf()` function):

```
pdf("file_path/file_name.pdf", width, height, ...)
  All plotting code should go here
dev.off()
```

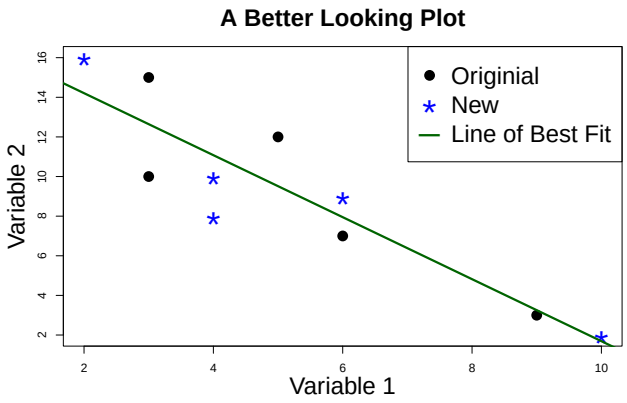
We end with the `dev.off()` function so **R** knows when the plotting is done.

Saving Plots Example

```
x <- c(3, 3, 5, 6, 9); y <- c(15, 10, 12, 7, 3)
pdf("better_plot.pdf", width=9, height=6)
plot(x, y, xlab = "Variable 1", ylab = "Variable 2",
     main = "A Better Looking Plot", pch = 16, cex = 2,
     cex.lab = 2, cex.main = 2, mgp = c(2.5,1,0),
     xlim = c(2, 10), ylim = c(2, 16))
xnew <- c(2, 4, 4, 6, 10); ynew <- c(16, 10, 8, 9, 2)
points(xnew, ynew, pch = "*", cex = 3, col = "blue")
abline(17.3377, -1.5649, col = "darkgreen", lwd = 3)
legend("topright", c("Originial", "New", "Line of Best Fit"),
       pch=c(16, 42, 95), cex=c(2, 2, 2), pt.cex=c(2, 3, 3),
       col = c("black", "blue", "darkgreen"))
dev.off()
```

```
pdf
  2
```


Saving Plots Example



Other Plots

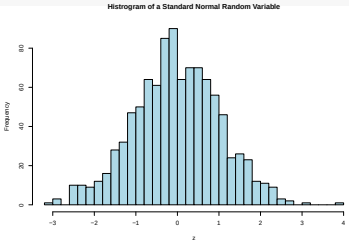
We mentioned that there are lots of other types of plots:

- `hist()`: used for making histograms for a quantitative variable.
- `boxplot()`: used for making box plots for a quantitative variable.
- `barplot()`: used for making bar plots for a categorical variable.
- `pie()`: used for making pie charts for a categorical variable (not recommended).

Many of these have similar syntax to the `plot()` function. As always, you can type "?" before the name of the function to pull up documentation on these functions.

Histogram Example

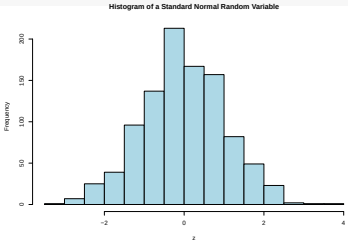
```
set.seed(1) # Sets the random seed so
            #   results are reproducible
z <- rnorm(1000) # Generates random values
hist(z, breaks = 30, col = "lightblue",
  main = "Histrogram of a Standard Normal Random Variable")
```



Histogram Example

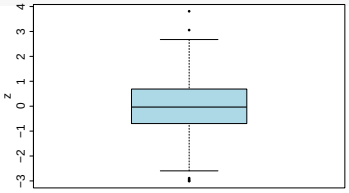
This is a histogram of the same data, but with a different binwidth.

```
hist(z, breaks = 15, col = "lightblue",
  main = "Histogram of a Standard Normal Random Variable")
```



Boxplot Example

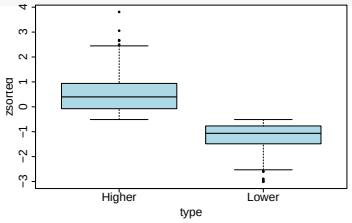
```
set.seed(1)
z <- rnorm(1000)

boxplot(z, cex.lab = 2, cex.axis = 2, ylab = "z",
        col = "lightblue", pch = 16)
```



Boxplot Example

Boxplots are most useful for making comparisons.

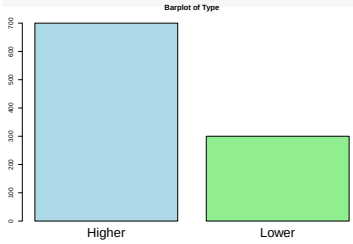
```
zsorted <- sort(z) # Arrange from least to greatest
type <- c(rep("Lower", 300), rep("Higher", 700))

# Splits z by type and make two boxplots
boxplot(zsorted ~ type, cex.lab = 2, cex.axis = 2,
        col = "lightblue", pch = 16)
```



Barplot Example

For barplots, we need to use the `table()` function to count the frequencies of each category for a categorical variable.

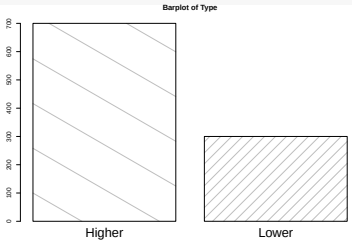
```
type <- c(rep("Lower", 300), rep("Higher", 700))
barplot(table(type), names = c("Higher", "Lower"),
        cex.names = 2, main = "Barplot of Type",
        col = c("lightblue", "lightgreen"))
```



Barplot Example

There is lots of customization you can do with barplots. For example the `density` option determines how close the lines in the barplot are to each other while the `angle` option will determine the angle of those lines.

```
barplot(table(type), names = c("Higher", "Lower"),
        cex.names = 2, main = "Barplot of Type",
        density = c(1, 5), angle = c(-30, 45))
```



Section 7

Graphing with ggplot2 (focus on this)

ggplot2 Introduction

While knowing how to plot using the base **R** packages is important, many **R** users are using the ggplot2 package (which is part of the tidyverse) more and more for making better-looking plots.

Advantages of ggplot2

- It's consistent! gg = "grammar of graphics"; easy base system for adding/removing plot elements, with room for being fancy too
- Very flexible
- Themes available to polish plot appearance
- Active maintenance/development = getting better all the time!
- It can do quick-and-dirty and complex, so you only need one system
- Plots, or whole parts of plots, can be saved as objects
- Easy to add complexity or revert to earlier plot

Introduction

Disadvantages of ggplot2

- Sometimes more complicated than base **R** plotting
- Difficult to work with in iterated functions
- No 3-D graphics
- ggplot is often slower than base graphics
- The default colors can be difficult to change
- You might need to change the structure of your data frame to make certain plots (use `tidyr::pivot_longer()`)

ggplot Basics

There are three primary components to plotting with ggplot2:

- The **data** component. This is what data set and variables we are actually plotting.
- The **geometry** component. This describes what it is we are plotting. Examples include barplots, scatter plots, histograms, smooth densities, qqplots, boxplots, etc.
- The **aesthetic mapping** or just the **mapping**. The two most important cues in this plot are the point positions on the x-axis and y-axis. Each point represents a different observation, and we map data about these observations to visual cues like x- and y-scale. Color is another visual cue that we map to region. How this is defined depends on what type of geometry we are using.

Example dataset: Diamonds

```
install.packages("ggplot2")
library(ggplot2)
```

Variable	Description	Values
price	price in US dollars	$326-$18,823
carat	weight of the diamond	0.2-5.01
cut	quality of the cut	Fair, Good, Very Good, Premium, Ideal
color	diamond color	J (worst) to D (best)
clarity	measurement of how clear the diamond is	I1 (worst), SI2, SI1, VS2, VS1, VVS2, VVS1, IF (best)
x	length in mm	0-10.74
y	width in mm	0-58.9
z	depth in mm	0-31.8
depth	total depth percentage	43-79
table	width of top of diamond relative to widest point	43-95

Figure 1: Diamonds

Example dataset: Diamonds

```
head(diamonds)
```

```
# A tibble: 6 x 10
  carat cut       color clarity depth table price     x
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl>
1  0.23 Ideal     E     SI2      61.5    55   326  3.95
2  0.21 Premium   E     SI1      59.8    61   326  3.89
3  0.23 Good      E     VS1      56.9    65   327  4.05
4  0.29 Premium   I     VS2      62.4    58   334  4.2
5  0.31 Good      J     SI2      63.3    58   335  4.34
6  0.24 Very Good J     VVS2     62.8    57   336  3.94
# i 2 more variables: y <dbl>, z <dbl>
```


Example dataset: Diamonds

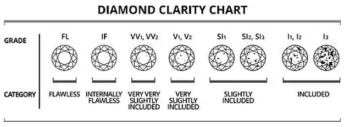


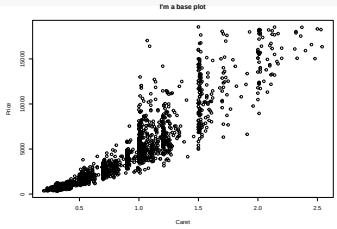
Figure 2: Diamond clarity is a measure of the purity and rarity of the stone, graded by the visibility of these characteristics under 10-power magnification. A stone is graded as flawless if, under 10-power magnification, no inclusions (internal flaws) and no blemishes (external imperfections) are visible.

- The dataset contains information about 53,940 round-cut diamonds
- There are 10 variables measuring various pieces of information about the diamonds.
- There are 3 variables with an ordered factor structure: cut, color, & clarity

Example in Base Plotting

There is essentially just one primary function to know: `ggplot()`. However, `ggplot()` needs lots of other basic functions.

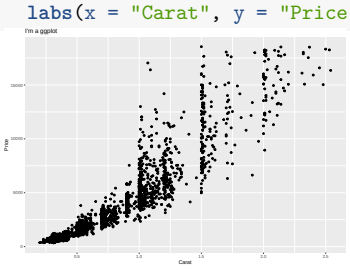
```
# load the diamonds dataset and take a sample of 2000
data(diamonds); set.seed(2023)
diam <- diamonds[sample(1:53940,2000),]
plot(diam$carat, diam$price, main = "I'm a base plot",
     xlab = "Caret", ylab = "Price")
```



Example in ggplot2

In a ggplot, we need to begin with the `ggplot()` function and then add on (literally with a + sign) to that plot using other commands. In this case, I put `geom_point()` to add those solid dots.

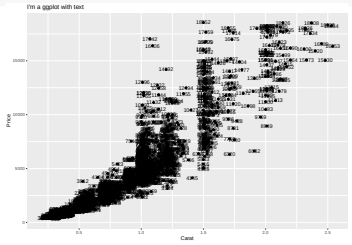
```
ggplot(data = diam) +
  geom_point(aes(x = carat, y = price)) +
  labs(x = "Carat", y = "Price", title = "I'm a ggplot")
```



Example in ggplot2

Here is an example adding text to the plot.

```
ggplot(data = diam) +
  geom_point(aes(x = carat, y = price)) +
  geom_text(aes(x = carat, y = price, label = price)) +
  labs(x = "Carat", y = "Price",
       title = "I'm a ggplot with text")
```



Global vs Local Aesthetic Mapping

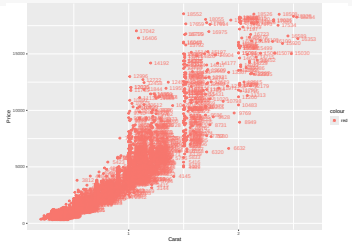
Instead of putting the x and y in the geom_() function, we can put it in the ggplot() and it will apply everywhere.

Anything put into the ggplot() function will apply globally to the entire plot whereas anything put into the geometry will only apply to that geometry. Some options, like size, can only be put into the geometry.

Example in ggplot2

This will make both the points and text red. The nudge_x option will move the text to the right 0.1 units.

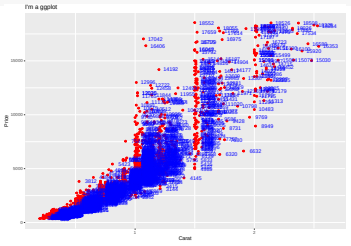
```
ggplot(data = diam, aes(x = carat, y = price, color = "red"))
  geom_point() +
  geom_text(aes(label = price), nudge_x = 0.1) +
  labs(x = "Carat", y = "Price")
```



Example in ggplot2

This will make the points red, but the text blue.

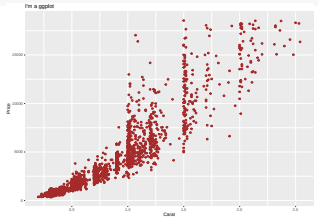
```
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_point(color = "red") +
  geom_text(aes(label=price), nudge_x = 0.1, color="blue") +
  labs(x = "Carat", y = "Price",
       title = "I'm a ggplot")
```



Piping in ggplot2

Pipes work very well with ggplot also. Remember that pipes are a part of the tidyverse (in thr dplyr package) and by default, piping puts the thing being piped into the first argument of the function.

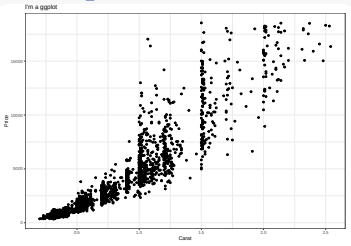
```
library(tidyverse)
diam |>
  ggplot(aes(carat, price)) + geom_point(col = "brown") +
  labs(x = "Carat", y = "Price", title = "I'm a ggplot")
```



Example in ggplot2

We can change the background using theme_...(). There are theme_bw(), theme_dark(), theme_classic(), and more.

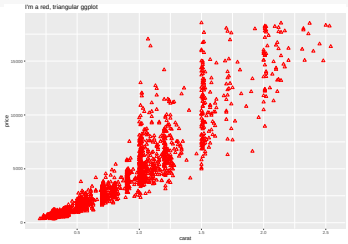
```
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_point() +
  labs(x = "Carat", y = "Price", title = "I'm a ggplot") +
  theme_bw()
```



Changing the Graph Options in ggplot2

We can change the type of points added in the `geom_point()` function.

```
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_point(size = 2, color = "red", shape = 2) +
  labs(title = "I'm a red, triangular ggplot")
```

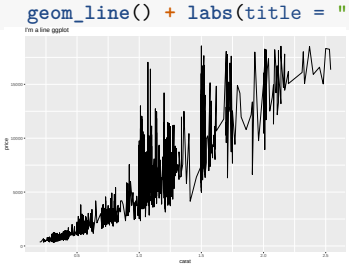


The shape argument works just like pch in base plotting.

Changing the Graph Type in ggplot2

Instead of adding `geom_point()`, we can add something else, like `geom_line()`

```
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_line() + labs(title = "I'm a line ggplot")
```

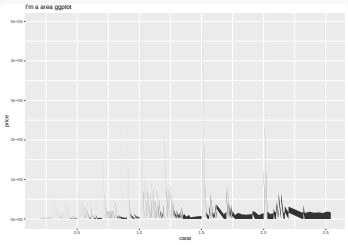


This looks strange for this plot, though.

Changing the Graph Type in ggplot2

Or even something like shading the area under the points.

```
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_area() + labs(title = "I'm a area ggplot")
```

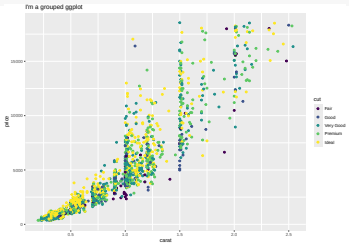


This looks even stranger in this case.

Grouping by Another Variable

ggplot makes it easy to split the data using another variable. Simply put the `col` argument in the `aes()` function in `ggplot`. This will automatically add a legend as well. We can change the shape based on another variable too.

```
ggplot(data = diam, aes(x = carat, y = price, col = cut)) +
  geom_point() + labs(title = "I'm a grouped ggplot")
```



Grouping by Another Variable

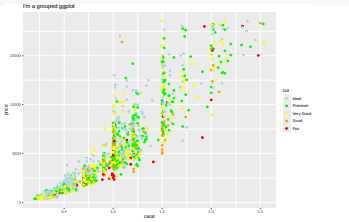
We can manually change the colors using the `values` option in the `scale_color_manual()` add on function.

```
ggplot(data = diam, aes(x = carat, y = price, col = cut)) +
  geom_point() + labs(title = "I'm a grouped ggplot") +
  scale_color_manual(values = c("red", "orange", "yellow",
                                "green","lightblue"))
```

Grouping by Another Variable

We can manually change the order of the categories in the legend using the `breaks` option in the `scale_color_manual()` add on function.

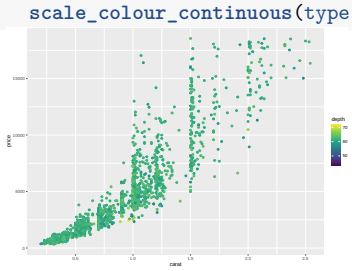
```
ggplot(data = diam, aes(x = carat, y = price, col = cut)) +
  geom_point() + labs(title = "I'm a grouped ggplot") +
  scale_color_manual(
    breaks = c("Ideal","Premium","Very Good","Good","Fair"),
    values = c("lightblue", "green","yellow","orange","red"))
```



Grouping by Another Variable

We can also group by a continuous variable. In this case, we can change the color based on the depth variable.

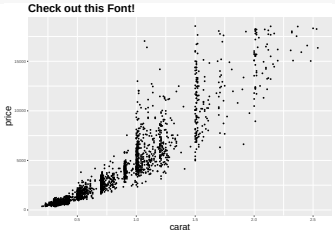
```
# color by a continuous variable
ggplot(data = diam, aes(x = carat,
          y = price, color = depth)) + geom_point() +
  scale_colour_continuous(type = "viridis")
```



Changing Font Size and Type

We can change font size and type in the `theme()` function:

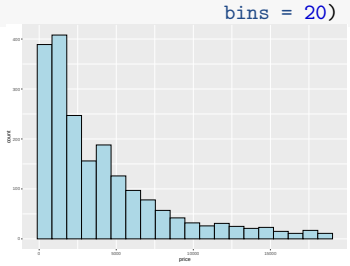
```
# color by a continuous variable
ggplot(data = diam, aes(x = carat, y = price)) +
  geom_point(shape = 16, size = 1.5) +
  labs(title = "Check out this Font!") +
  theme(axis.title = element_text(size = 20),
        plot.title = element_text(size = 24, face = "bold"))
```



ggplot2 for a Single Quantitative Variable: Histogram

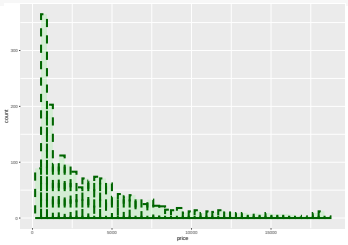
Of course, we can also use `ggplot()` for plotting a single variable. We can make histograms, boxplots, dotplots, etc.

```
# Save the base plot as an object p. Then add to p.
p <- ggplot(data = diam, aes(x = price))
p + geom_histogram(color = "black", fill = "lightblue",
                   bins = 20)
```



ggplot2 for a Single Quantitative Variable: Histogram

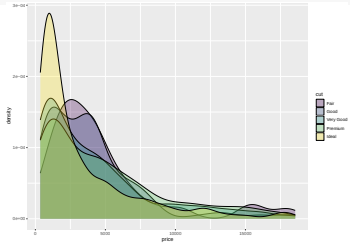
```
# We already created p, so we can add other options to it.
# Smaller alpha makes the plot more transparent.
p + geom_histogram(color = "darkgreen", fill = "green",
                   bins = 50, alpha = 0.1, lty = 2, lwd = 2)
```



ggplot2 for a Single Quant. Variable: Layered Histograms

A layered histogram is a good way to compare the distribution of a variable across groups. `geom_histogram` works well for two groups, but geom_density is easier to look at for several groups.

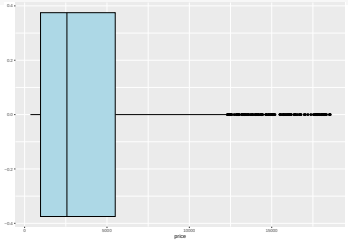
```
ggplot(data = diam, aes(x = price, fill = cut)) +
  geom_density(alpha = 0.3)
```



ggplot2 for a Single Quantitative Variable: Boxplot

Creating a basic boxplot. We can also make it vertical by putting `y = price` in the `aes()` function instead of `x = price`.

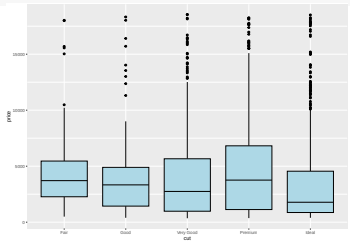
```
ggplot(data = diam, aes(x = price)) +
  geom_boxplot(color = "black", fill = "lightblue")
```



ggplot2 for a Single Quant. Variable: Side-by-Side Boxplots

We can make side-by-side boxplots grouped by a categorical variable as x (or as y if you want side-by-side horizontal boxplots).

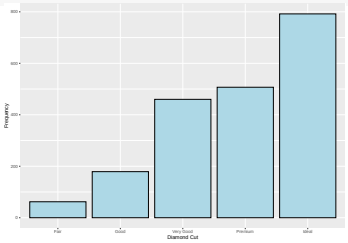
```
ggplot(data = diam, aes(x = cut, y = price)) +
  geom_boxplot(color = "black", fill = "lightblue")
```



ggplot2 for a Single Categorical Variable: Barplot

We can make plots for categorical variables as well.

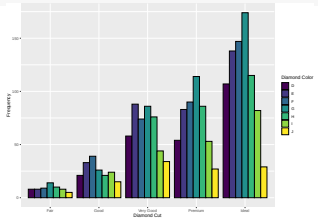
```
ggplot(data = diam, aes(x = cut)) +
  geom_bar(color = "black", fill = "lightblue") +
  labs(x = "Diamond Cut", y = "Frequency")
```



ggplot2 for a Single Categ. Variable: Side-by-Side Barplots

We can split the bars by another variable. In this case, we will make a plot of diamond cut and break it up by diamond color and put the bars side by side.

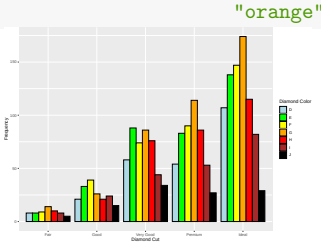
```
ggplot(data = diam, aes(x = cut, fill = color)) +
  geom_bar(color = "black", position = "dodge") +
  labs(x = "Diamond Cut", y = "Frequency",
       fill = "Diamond Color")
```



ggplot2 for a Single Categ. Variable: Side-by-Side Barplots

We can change the fill colors using `scale_fill_manual()`.

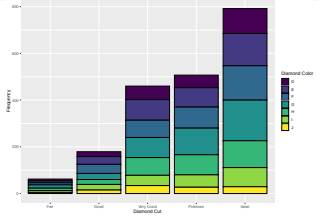
```
ggplot(data = diam, aes(x = cut, fill = color)) +
  geom_bar(color = "black", position = "dodge") +
  labs(x = "Diamond Cut", y = "Frequency",
       fill = "Diamond Color") +
  scale_fill_manual(values = c("lightblue","green","yellow",
                               "orange","red","brown","black"))
```



ggplot2 for a Single Categorical Variable: Stacked Barplot

We can split the bars by another variable. In this case, we will make a plot of diamond cut and break it up by diamond color and stack the bars.

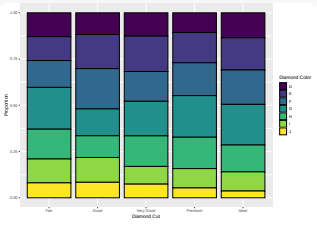
```
ggplot(data = diam, aes(x = cut, fill = color)) +
  geom_bar(color = "black", position = "stack") +
  labs(x = "Diamond Cut", y = "Frequency",
       fill = "Diamond Color")
```



ggplot2 for a Single Categorical Variable: Stacked Barplot

We can split the bars by another variable. In this case, we will make a plot of diamond cut and break it up by diamond color, stack the bars, and adjust them so each bar totals 100%.

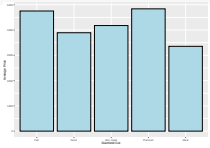
```
ggplot(data = diam, aes(x = cut, fill = color)) +
  geom_bar(color = "black", position = "fill") +
  labs(x = "Diamond Cut", y = "Proportion",
       fill = "Diamond Color")
```



ggplot2 Barplot Identity

We often want to use a column of a data frame or tibble as the heights of our bar plot instead of having ggplot tabulate them for us. For this, we need to put `stat = identity` in the `geom_bar()` function.

```
diam |> group_by(cut) |>
  summarize(avg_price = mean(price)) |>
  ggplot(aes(x = cut, y = avg_price)) +
  geom_bar(color = "black", stat = "identity",
           fill = "lightblue") +
  labs(x = "Diamond Cut", y = "Average Price")
```



Using ggplot2 for more complex plots

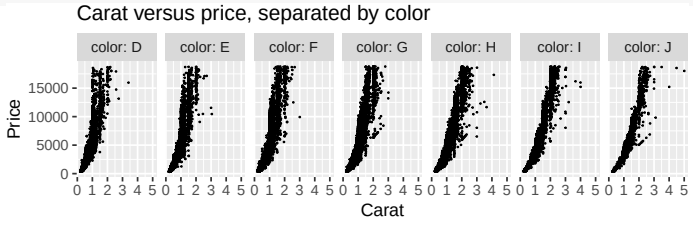
Use facet_grid() or facet_wrap() to create a separate plot for each value of a factor variable. We don't have to change any of the original plotting code, just add the facet command to it. Faceting can also be done on more than one categorical variable to create a grid of plots.

Additionally, it is sometimes helpful to save a simpler version of a plot, and then add onto it later with additional layers (for example, an if/else statement that plots different layers dependent on if a criterion is met or not).

We might want to summarize the data in the previous plot with a smoother on top of the points. With ggplot, we can simply add the `geom_smooth()` command. Each geom just adds another layer to the plot.

Using ggplot2 for more complex plots

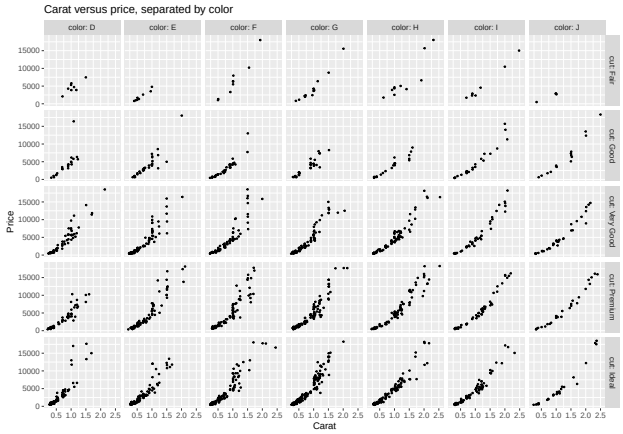
```
# make the basis for a plot using ggplot save it as p
p <- ggplot(data = diamonds, aes(x = carat, y = price))
# add a geom (points) and display the plot
p + geom_point(size=0.1) + facet_grid(cols = vars(color),
     labeller = label_both) + labs(x = "Carat", y = "Price",
       title = "Carat versus price, separated by color")
```



Using ggplot2 for more complex plots

```
# make the basis for a plot using ggplot save it as p
p <- ggplot(data = diam, aes(x = carat, y = price))
# add a geom (points) and display the plot
p + geom_point(size=0.1) + facet_grid(rows = vars(cut),
       cols = vars(color), labeller = label_both) +
  labs(x = "Carat", y = "Price",
       title = "Carat versus price, separated by color")
```


Using ggplot2 for more complex plots



Summary

The syntax of a ggplot is `ggplot(data, aes(x, y))` and you add on to the plot with + at the end of each line.

The most useful functions to add onto a ggplot are:

- `geom_point()`, `geom_line()`, `geom_histogram()`, `geom_boxplot()`, `geom_text()`, etc.
- `labs()` for labels including a plot title.
- `lims()` for limits.
- `theme()` for text size and visually changing other things.
- `scale_color_manual()` or `scale_fill_manual()` for changing the color or fill of the plot manually.

Further Resources & Assistance

- Cheat sheet for data visualization with ggplot2 (accessible in Rstudio by going to Help -> Cheat Sheets -> Data visualization with ggplot2)
- ggplot2 documentation
- Google
- Stack overflow
- Hadley Wickham's book: https://ggplot2-book.org/
- Rafael Irizarry's book: http://rafalab.dfci.harvard.edu/dsbook-part-1/dataviz/ggplot2.html.
- Note: these slides were adapted from slides created by Aubrey Odom.

Session info

```
sessionInfo()
```

```
R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.7.2

Matrix products: default
BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBL
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK version 3.12

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Denver
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets
[6] methods   base

other attached packages:
 [1] knitr_1.51      dslabs_0.9.1    lubridate_1.9.4
 [4] forcats_1.0.1   stringr_1.6.0   dplyr_1.1.4
 [7] purrr_1.2.0     readr_2.1.6     tidyr_1.3.2
[10] tibble_3.3.0    ggplot2_4.0.1   tidyverse_2.0.0

loaded via a namespace (and not attached):
 [1] gtable_0.3.6       jsonlite_2.0.0
 [3] compiler_4.5.2     tinytex_0.58
 [5] tidyselect_1.2.1   scales_1.4.0
 [7] yaml_2.3.12        fastmap_1.2.0
 [9] R6_2.6.1           labeling_0.4.3
[11] generics_0.1.4     pillar_1.11.1
[13] RColorBrewer_1.1-3 tzdb_0.5.0
```